Part 1: Local Docker Setup & Mounting Local GeoJSON

The repository includes a ready-to-run multi-stage `Dockerfile`, `nginx.conf`, and `docker-compose.yml`.

1. How Docker Reads Your Local GeoJSON File

The `docker-compose.yml` mounts a local directory (`./data`) directly into the container's web server at `/usr/share/nginx/html/data`:

1. In the same directory where you extract the zip, create a `data` folder:

   code Bash

   ```
   mkdir -p data
   ```

2. Put any GeoJSON file named `signals.geojson` inside that folder:

   code Bash

   ```
   cp /path/to/your/file.geojson ./data/signals.geojson
   ```

3. When you start Docker, the app automatically detects and loads `http://localhost:3000/data/signals.geojson` on startup.
4. If you update the file on your local machine, the built-in **Live Shared Feed & Directory Sync** toolbar automatically polls and updates it with zero restart required.

---

2. Running with Docker Compose (Recommended)

code Bash

```
# 1. Unzip and enter the project folder
unzip osint-app.zip -d osint-app
cd osint-app

# 2. Put your GeoJSON file in ./data/signals.geojson
mkdir -p data
cp /your/local/signals.geojson data/signals.geojson

# 3. Build and launch container
docker compose up --build -d

# 4. Open in browser
# http://localhost:3000
```

To stop:

code Bash

```
docker compose down
```

---

3. Running with Pure Docker CLI (Alternative)

code Bash

```bash
# 1. Build the Docker image
docker build -t osint-signals-map .

# 2. Run the container with your local path mounted:
docker run -d \
  --name osint-signals-map \
  -p 3000:3000 \
  -v "$(pwd)/data:/usr/share/nginx/html/data:ro" \
  osint-signals-map
```

(Replace `$(pwd)/data` with any absolute path on your host machine containing `signals.geojson`).

Part 2: LLM Contact Points & Model Agnosticism

1. Are there active LLM calls in this App?

No. At runtime, this application makes zero external LLM calls.

- **Architecture:** The application operates as a **100% deterministic, client-side, zero-API system**. It is completely air-gapped capable, produces zero cloud API costs, and leaks no intelligence data to external servers.
- **Why `@google/genai` is in `package.json`:** It was included as part of the default workspace package dependencies, but no active imports or API calls are used in the codebase.
- **How synthesis and intelligence analysis work:**
  - **Strategic Briefing & Convergence (`BriefingGenerator.tsx`)**: Analyzes domain overlaps (Tech

    Regulatory Economic), geographic clusters, and high-relevance signals algorithmically in TypeScript.

  - **OSINT Strategic Assessment (`SignalDossier.tsx`)**: Derives domain implications, situational summaries, and impact vectors directly from signal metadata tags.
  - **Impact Matrix (`ImpactMatrix.tsx`)**: Computes sector impact distributions deterministically from GeoJSON properties.

---

2. Is the App Model-Agnostic?

Yes, completely model-agnostic. If you want to connect an LLM backend (for example, to generate conversational summaries or extended foresight paragraphs):

- You do **not** have to use Gemini.
- You can plug in **Ollama** (`llama3.3:70b`, `qwen2.5:32b`, `mistral`), **vLLM**, **OpenAI**, **Anthropic**, or **Gemini**.
- Because the input data is standard GeoJSON (`FeatureCollection`), any LLM that accepts JSON or Markdown can consume it.

---

3. Recommended Models by Capability (For Local or Private Deployments)

If you replace the deterministic briefing generator with a live LLM integration, here are the exact capacity recommendations matching each feature:

Feature / Contact Point	Purpose / Task Type	Recommended Gemini Model	Recommended Local/Self-Hosted Model (Ollama/vLLM)	Recommended OpenAI/Anthropic Alternative
OSINT Intelligence Briefing (`BriefingGenerator.tsx`)	Multi-signal correlation, regulatory cross-referencing, convergence mapping	`gemini-2.5-flash` or `gemini-1.5-pro`	**Llama 3.3 (70B)** or **Qwen 2.5 (32B/72B)**	`gpt-4o-mini` or `claude-3-5-sonnet`
Tactical Signal Dossier (`SignalDossier.tsx`)	Single-signal impact analysis and executive summary generation	`gemini-2.5-flash`	**Mistral Nemo (12B)** or **Llama 3.1 (8B)**	`gpt-4o-mini`
Geospatial Coordinate Extraction (if automating unmapped items)	Natural language address to lat/lng JSON structured output	`gemini-2.5-flash`	**Qwen 2.5 (7B/14B)**	`gpt-4o-mini`

Files Created in the Repository

1. `Dockerfile`: Optimized multi-stage build (`node:20-alpine` builder `nginx:alpine` runtime on port 3000).
2. `nginx.conf`: Nginx configuration with Single-Page Application routing fallback, gzip compression, and `/data/` route with no-cache headers for live local file changes.
3. `docker-compose.yml`: Automated service configuration with bind-mount volume for `./data`.
4. `.dockerignore`: Keeps Docker build context lean by ignoring `node_modules` and build directories.